LAPORAN PRAKTIKUM

INTERNET OF THINGS II

Protokol Komunikasi Telemetri Industri (MQTT)



Oleh:

NAMA : Albertus Satriyo Nugroho
NPM 25782002
KELAS : 3A
MATKUL : Internet of Things (IOT)

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.
Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026

1: Publikasi Data Sensor Lingkungan (DHT) Berformat JSON

Tugas5.7.ino

```ino
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// Kredensial Jaringan Wi-Fi (Gunakan Hotspot Pribadi Anda)
const char* ssid = "Infinix GT 30 Pro";
const char* password = "murahcpz";

// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;

// IDENTITAS KLIEN (Wajib Unik! Ganti dengan NIM Anda agar tidak bertabrakan)
const char* clientId = "ESP8266_LabIoT_25782002";

// Definisi Topik Publikasi Telemetri Mandiri
```

Output | Serial Monitor

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4') | New Line | 115200 baud

```
Wi-Fi Terkoneksi!
Alamat IP Lokal: 10.158.168.188
Menghubungkan ke MQTT Broker... Sukses Terhubung!
Telemetri Terkirim [kampus/labiot/25782002/telemetri]: {"node_id":"ESP8266_LabIoT_25782002","suhu":2.00,"kelembapan":17.70}
Telemetri Terkirim [kampus/labiot/25782002/telemetri]: {"node_id":"ESP8266_LabIoT_25782002","suhu":2.00,"kelembapan":17.70}
```

MQTT Explorer

Application Edit View

MQTT Explorer | Search... | DISCONNECT

- broker.hivemq.com
  - kampus
    - labiot
      - 25782002
        - telemetri = {"node_id":"ESP8266_LabIoT_25782002","suhu":1.90,"kelembapan":17.60}

Topic

kampus / labiot / 25782002 / telemetri

Value

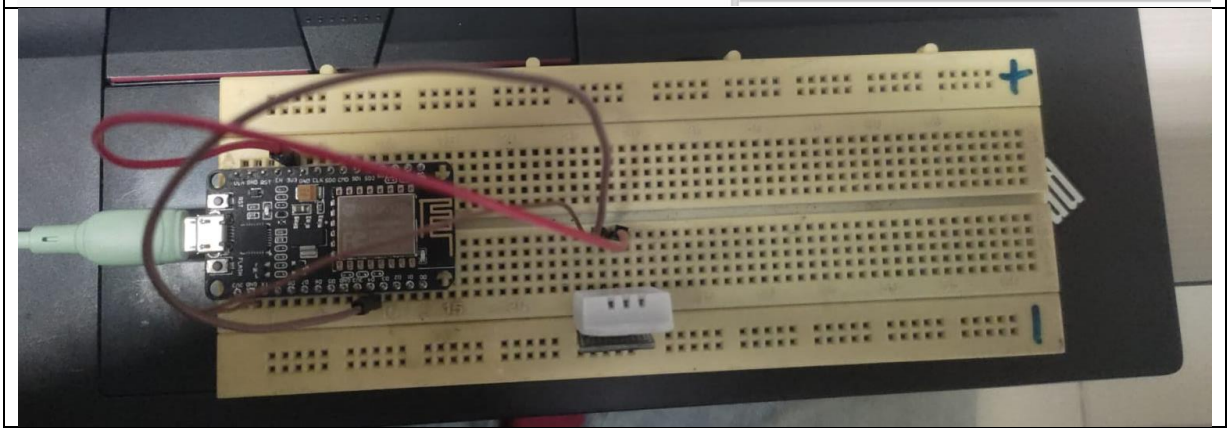
QoS: 0

12/10/2026 09:59:01

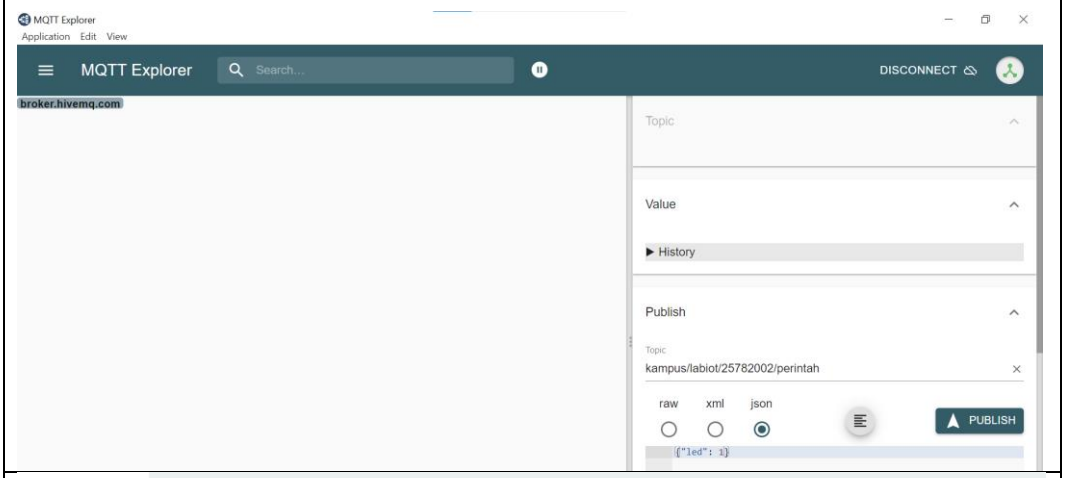
```
{
  "node_id": "ESP8266_LabIoT_25782002",
  "suhu": 1.9,
  "kelembapan": 17.5
  "kelembapan": 17.6
}
```

Comparing with **previous** message: + 1 line, - 1 line

▶ History 3



2: Subskripsi dan Kontrol Indikator LED Asinkron



Tugas5.7.ino

```
#include <ArduinoJson.h>

// Kredensial Jaringan Wi-Fi
const char* ssid = "Infinix GT 30 Pro";
const char* password = "murahcpz";

// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_25782002_Sub";

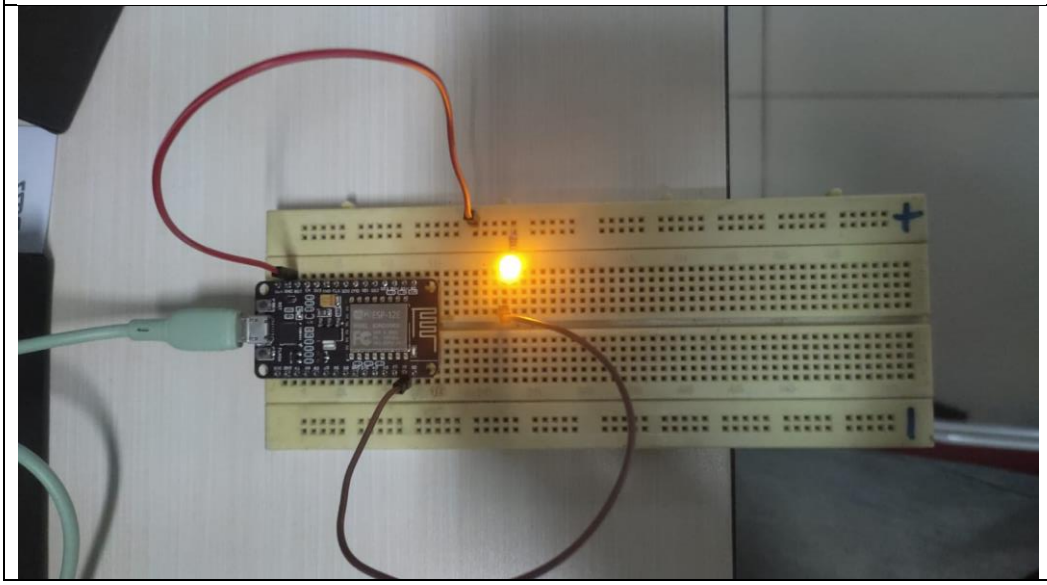
// Definisi Topik Subskripsi Perintah Mandiri
const char* topicSubCommand = "kampus/labiot/25782002/perintah";

// Konfigurasi Pin Indikator LED
const byte ledPin = 5; // Pin D1 (GPIO 5)
```

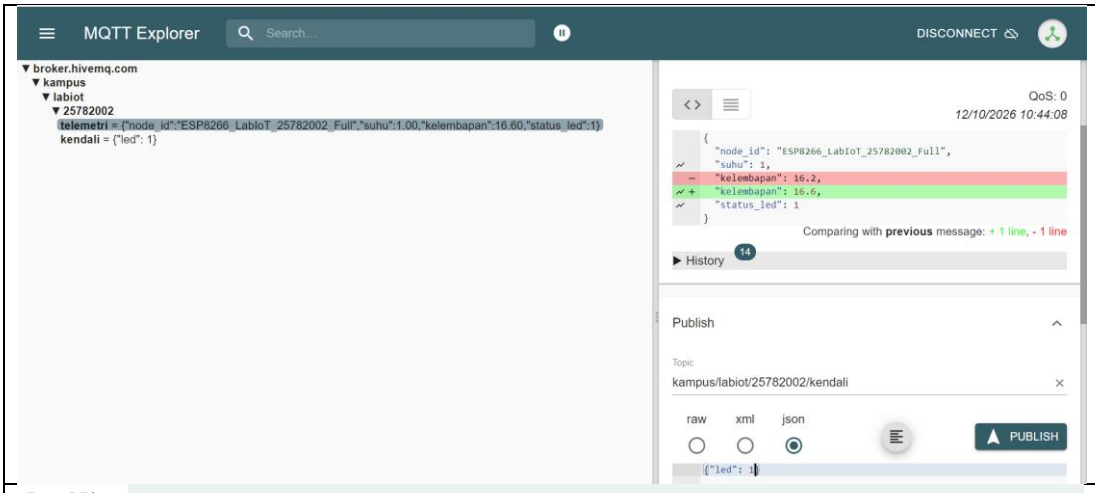
Output Serial Monitor ×

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')

```
Payload Diterima: {"led": 0}
--> Aksi: Status LED diubah menjadi PADAM (OFF)
========================================
========================================
Pesan Baru Masuk pada Topik: kampus/labiot/25782002/perintah
Payload Diterima: {"led": 1}
--> Aksi: Status LED diubah menjadi MENYALA (ON)
========================================
```



3: Sistem Terpadu Dua Arah (Full Duplex Telemetry & Control)



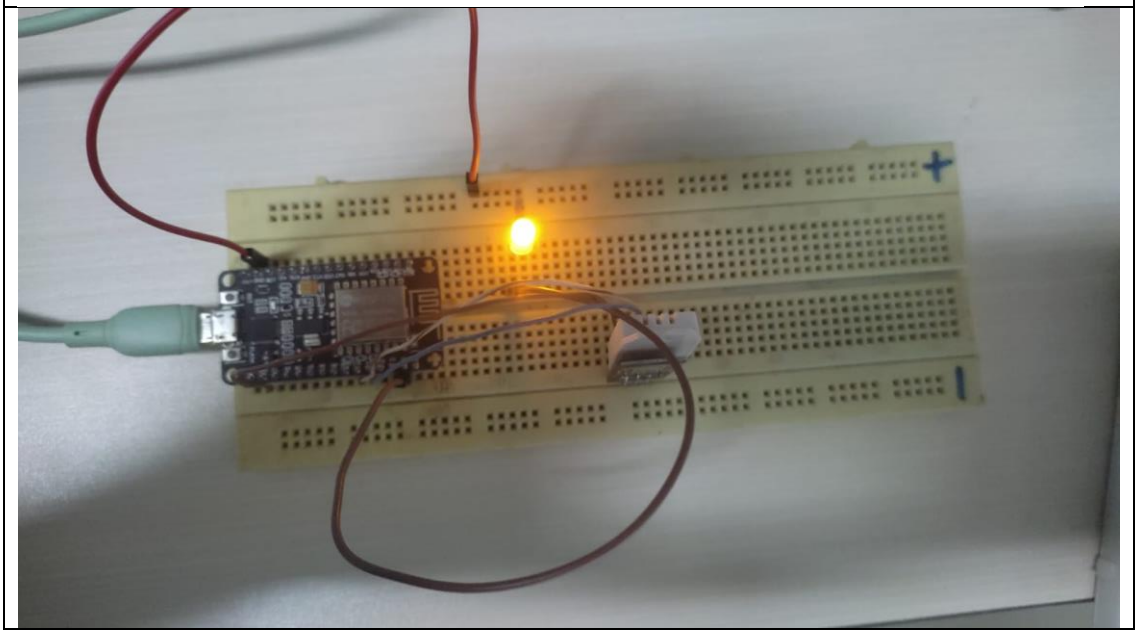
Tugas5.7.ino

```
11  const char* mqttServer = "broker.hivemq.com";
12  const int mqttPort = 1883;
13  const char* clientId = "ESP8266_LabIoT_25782002_Full";
14
15  // Pembagian Topik Mandiri
16  const char* topicPublish   = "kampus/labiot/25782002/telemetri";
17  const char* topicSubscribe = "kampus/labiot/25782002/kendali";
18
19  // Alokasi Pin Perangkat Keras
20  const byte dhtPin = 2; // Pin D4 (GPIO 2)
21  const byte ledPin = 5; // Pin D1 (GPIO 5)
22  #define DHTTYPE DHT11
23
24  DHT dht(dhtPin, DHTTYPE);
25  WiFiClient espClient;
26  PubSubClient client(espClient);
27
```

Output | Serial Monitor

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4') | New Line

```
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782002_Full","suhu":1.00,"kelembapan":16.70,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782002_Full","suhu":1.00,"kelembapan":16.60,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782002_Full","suhu":1.00,"kelembapan":16.90,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_25782002_Full","suhu":1.00,"kelembapan":17.00,"status_led":0}
----------------------------------------
[KONTROL DITERIMA] Topik: kampus/labiot/25782002/kendali | Payload: {"led": 1}
--> [INDIKATOR] Status LED diubah: MENYALA
----------------------------------------
```



6.6 Latihan

1 (Analisis Arsitektur & Perbandingan Protokol):
Pada Modul 3, kita menggunakan protokol HTTP untuk membaca dan mengontrol perangkat. Jelaskan secara mendalam mengapa protokol MQTT jauh lebih unggul daripada HTTP dalam hal:
1. Efisiensi penggunaan kapasitas *bandwidth* pada paket data seluler.
2. Beban komputasi mikrokontroler (*CPU overhead*) saat menangani pertukaran data dua arah
3.Kemampuan pengiriman data secara simultan ke banyak perangkat penerima (*1-to-N distribution*

JAWAB:

1. Efisiensi Penggunaan Bandwidth pada Paket Data Seluler

Protokol MQTT (Berbasis Biner & Event-Driven): MQTT dirancang khusus dari awal sebagai protokol biner yang sangat ringkas, di mana ukuran *fixed header*-nya hanya 2 byte. Dengan memanfaatkan koneksi TCP yang persisten (*persistent connection*), MQTT tidak perlu melakukan *handshake* berulang-ulang pada setiap pengiriman data. Transmisi data bersifat *event-driven* (data hanya dikirimkan saat terjadi perubahan nilai atau kejadian tertentu), sehingga konsumsi kuota data seluler ditekan seminimal mungkin.

Sedangkan HTTP adalah protokol *request-response* yang bersifat *text-based*. Setiap kali permintaan dikirimkan, HTTP menyertakan *header* yang sangat besar (mulai dari ratusan hingga ribuan byte untuk memuat metadata seperti *User-Agent*, *Host*, *Cookies*, *Authorization*, dan *Accept*). Selain itu, untuk membaca status terbaru dari perangkat, HTTP umumnya mengandalkan mekanisme *polling* (mengirim permintaan secara berkala dalam interval waktu tertentu). Akibatnya, bandwidth seluler terus tersedot meskipun nilai data di lapangan sama sekali tidak mengalami perubahan.

2. Beban Komputasi Mikrokontroler (*CPU Overhead*) Saat Pertukaran Data Dua Arah

Protokol MQTT:
Memiliki beban komputasi (*CPU overhead*) yang sangat rendah karena struktur paketnya berbasis biner. Proses *encoding* dan *decoding* data dapat diselesaikan dengan instruksi CPU yang sangat sedikit dan cepat. Efisiensi pengelolaan memori ini membuat mikrokontroler dapat menghemat penggunaan daya listrik secara signifikan, sehingga perangkat dapat bekerja lebih lama dengan baterai dan mengoptimalkan mode *deep sleep*.

Protokol HTTP:
Membebankan tugas berat pada mikrokontroler (seperti ESP8266 atau ESP32) karena perangkat harus melakukan *parsing string* teks yang panjang, memproses format payload yang besar (seperti JSON), serta mengelola pembukaan dan penutupan soket jaringan secara konstan karena sifat HTTP yang *stateless*. Proses ini menguras siklus CPU dan memori RAM yang sangat terbatas pada mikrokontroler.

3. Kemampuan Pengiriman Data Secara Simultan ke Banyak Perangkat Penerima (Distribusi 1-to-N)

Protokol MQTT (Arsitektur Publish-Subscribe via Broker): MQTT menggunakan perantara terpusat yang disebut Broker (seperti Mosquitto).

Pengirim (*Publisher*) cukup mengirimkan satu pesan tunggal ke Broker pada topik tertentu (misalnya sistem/kontrol/lampu).
Broker secara otomatis menangani mekanisme *fan-out* untuk menyebarkan pesan tersebut ke seluruh perangkat penerima (*Subscribers*) yang berlangganan topik yang sama—baik itu puluhan, ratusan, maupun ribuan perangkat—secara simultan, instan, dan tanpa membebani sumber daya pengirim.

Latihan 2 (Analisis Hierarki Topik & Karakter Wildcard):
Sebuah pabrik manufaktur memiliki 3 lini produksi (*line1*, *line2*, *line3*), di mana setiap lini memiliki 4 mesin (*m1*, *m2*, *m3*, *m4*), dan setiap mesin melaporkan metrik suhu, getaran, dan status. Struktur topiknya adalah pabrik/<line>/<mesin>/<sensor>. Tentukan ekspresi topik subskripsi (*subscription topic*) yang tepat menggunakan karakter *wildcard* (+ atau #) untuk:

1.Memantau parameter suhu dari seluruh mesin di semua lini produksi.
2.Memantau seluruh parameter sensor (suhu, getaran, status) khusus untuk mesin m1 di lini line2.
3.Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan seluruh mesin di pabrik tersebut.

JAWAB:

1. Memantau parameter suhu dari seluruh mesin di semua lini produksi
   - Ekspresi Topik: pabrik/+/+/suhu
   - Penjelasan: Karakter + pertama menggantikan posisi semua lini produksi (line1, line2, line3), karakter + kedua menggantikan posisi semua mesin (m1 sampai m4), dan spesifik menargetkan sensor suhu.

2. Memantau seluruh parameter sensor khusus untuk mesin m1 di lini line2
   - Ekspresi Topik: pabrik/line2/m1/+
   - Penjelasan: Jalur lini dan mesin dikunci secara spesifik pada line2 dan m1, sedangkan level terakhir menggunakan + untuk mencakup seluruh parameter sensor secara bersamaan (suhu, getaran, status).

3. Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan mesin
   - Ekspresi Topik: pabrik/#
   - Penjelasan: Menggunakan *multi-level wildcard* (#) di akhir *root* topik pabrik/ untuk menangkap seluruh data, level hierarki, dan sub-topik apa pun yang berada di bawah direktori pabrik tanpa terkecuali.

Latihan 3 (Implementasi Serialisasi Data Dinamis):
Modifikasi blok penyusunan JSON pada Praktikum 1 agar paket telemetri menyertakan dua parameter tambahan berikut:

1."uptime_detik": Lama waktu aktif sistem dalam satuan detik (dihitung dari rumus millis() / 1000).
2."ip_lokal": Alamat IP lokal mikrokontroler dalam format string (WiFi.localIP().toString()).
Tuliskan baris modifikasi kode C++ yang Anda tambahkan pada fungsi pembentukan JSON!

JAWAB:

```
// Menambahkan parameter uptime sistem (dihitung dari millis() dibagi 1000)
doc["uptime_detik"] = millis() / 1000;

// Menambahkan parameter alamat IP lokal mikrokontroler dalam format string
doc["ip_lokal"] = WiFi.localIP().toString();
```

Latihan 4 (Analisis Mekanisme Reconnect & Loop Blocking):
Pada Praktikum 3, di dalam fungsi loop() terdapat pemanggilan rutin client.loop().
1. Apa konsekuensi teknis yang terjadi terhadap responsivitas penerimaan perintah *callback* jika seorang pengembang menyisipkan instruksi delay(6000) di dalam fungsi loop()?
2. Jelaskan bagaimana fungsi reconnect() bekerja memulihkan koneksi saat router Wi-Fi atau broker MQTT sempat terputus sesaat!

JAWAB:

1. Konsekwensinya:
- Keterlambatan Respons Perintah (*Callback*): Fungsi client.loop() bertanggung jawab untuk mendengarkan dan memproses paket data masuk dari broker MQTT. Karena client.loop() tidak terpanggil selama 6 detik, perintah kendali yang dikirim dari aplikasi/dashboard akan tertunda diproses (*latency* tinggi).

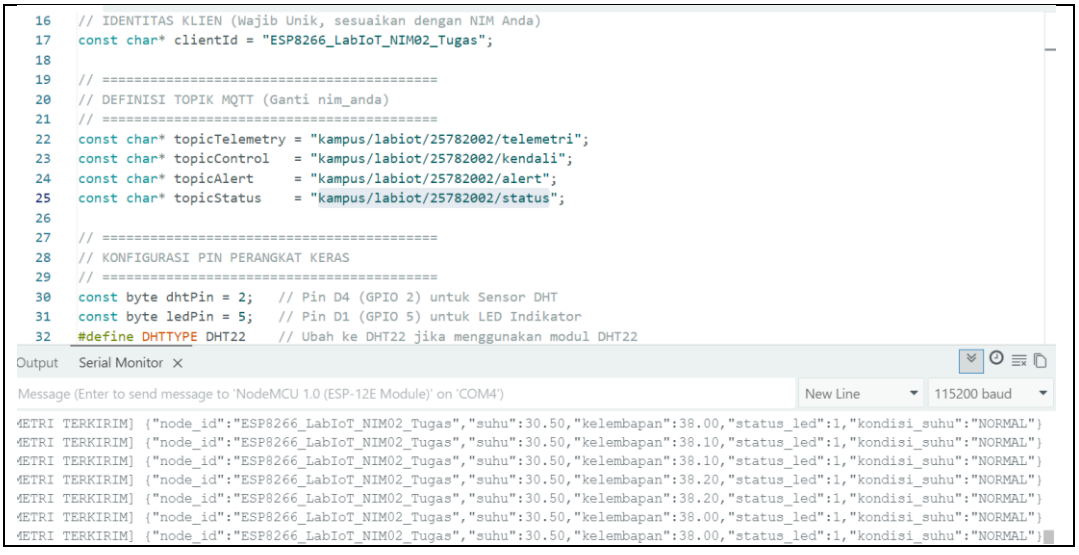
- Risiko Pemutusan Koneksi oleh Broker (*Keep-Alive Timeout*): MQTT memiliki mekanisme *Keep-Alive* di mana klien harus mengirimkan paket *ping* (*PINGREQ*) secara berkala untuk memberi tahu broker bahwa perangkat masih aktif. Jika delay(6000) membuat pengiriman *ping* terlewat dari batas waktu yang ditentukan, broker akan menganggap mikrokontroler telah mati (*offline*) dan secara sepihak memutuskan koneksi (*disconnect*).

2. cara kerjanya Meliputi:
1.Pemeriksaan Status Koneksi: Fungsi secara rutin mengecek apakah status klien saat ini bernilai terputus (!client.connected()).

2.Percobaan Koneksi Ulang (*Connection Attempt*): Jika terputus, program akan memanggil metode client.connect() untuk mencoba terhubung kembali ke broker MQTT menggunakan ID klien, *username*, dan *password* (jika ada).

3.Berlangganan Ulang Topik (*Re-Subscribe*): Begitu koneksi berhasil dipulihkan (client.connected() bernilai true), fungsi langsung mengeksekusi ulang perintah client.subscribe() untuk berlangganan kembali ke topik-topik sensor atau kontrol yang sebelumnya terputus akibat hilangnya sesi komunikasi**.**

6.7 Tugas: Sistem Telemetri & Kendali Closed-Loop Industri

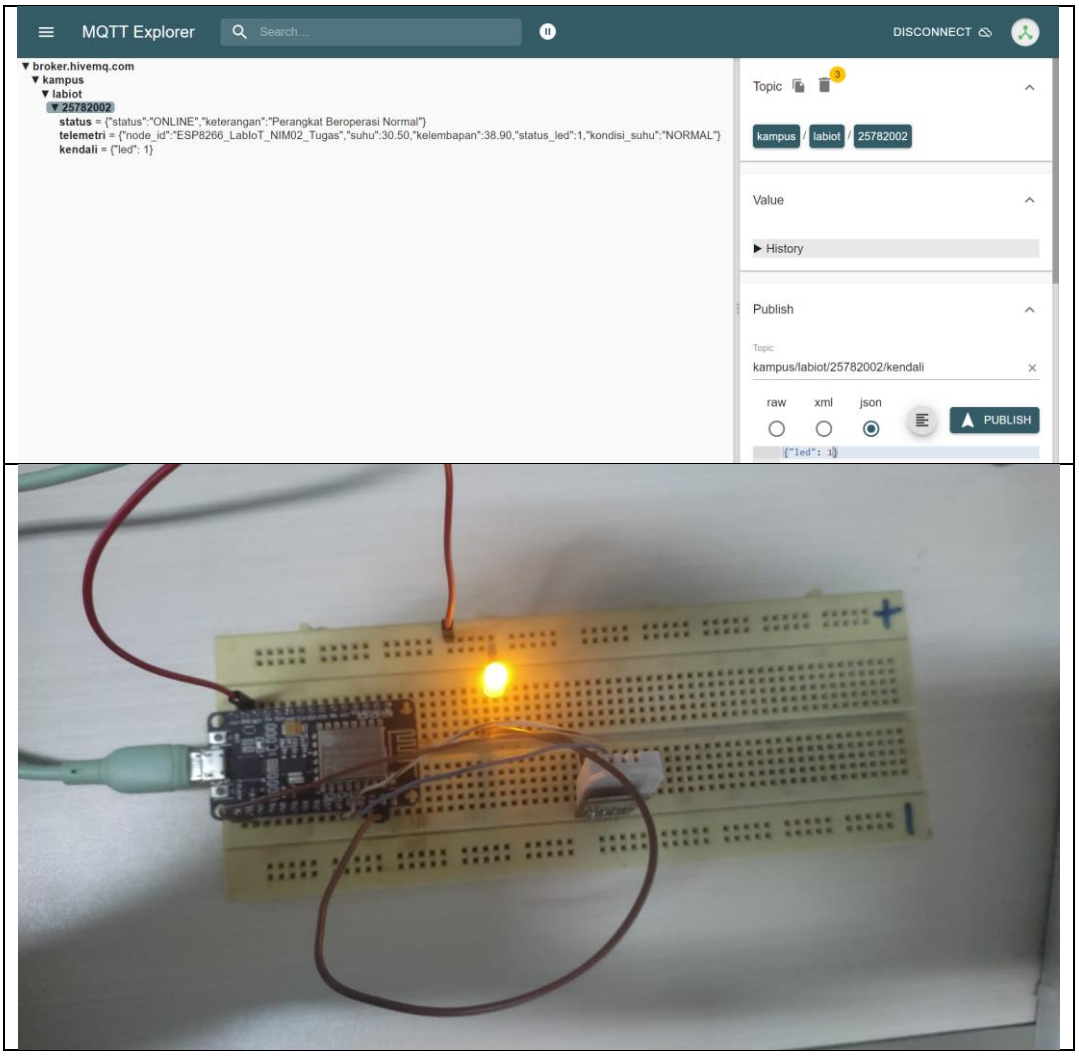


```
16  // IDENTITAS KLIEN (Wajib Unik, sesuaikan dengan NIM Anda)
17  const char* clientId = "ESP8266_LabIoT_NIM02_Tugas";
18
19  // ============================================
20  // DEFINISI TOPIK MQTT (Ganti nim_anda)
21  // ============================================
22  const char* topicTelemetry = "kampus/labiot/25782002/telemetri";
23  const char* topicControl   = "kampus/labiot/25782002/kendali";
24  const char* topicAlert     = "kampus/labiot/25782002/alert";
25  const char* topicStatus    = "kampus/labiot/25782002/status";
26
27  // ============================================
28  // KONFIGURASI PIN PERANGKAT KERAS
29  // ============================================
30  const byte dhtPin = 2;   // Pin D4 (GPIO 2) untuk Sensor DHT
31  const byte ledPin = 5;   // Pin D1 (GPIO 5) untuk LED Indikator
32  #define DHTTYPE DHT22    // Ubah ke DHT22 jika menggunakan modul DHT22
```

Output Serial Monitor ×

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4') New Line 115200 baud

```
ETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM02_Tugas","suhu":30.50,"kelembapan":38.00,"status_led":1,"kondisi_suhu":"NORMAL"}
ETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM02_Tugas","suhu":30.50,"kelembapan":38.10,"status_led":1,"kondisi_suhu":"NORMAL"}
ETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM02_Tugas","suhu":30.50,"kelembapan":38.10,"status_led":1,"kondisi_suhu":"NORMAL"}
ETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM02_Tugas","suhu":30.50,"kelembapan":38.20,"status_led":1,"kondisi_suhu":"NORMAL"}
ETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM02_Tugas","suhu":30.50,"kelembapan":38.20,"status_led":1,"kondisi_suhu":"NORMAL"}
ETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM02_Tugas","suhu":30.50,"kelembapan":38.00,"status_led":1,"kondisi_suhu":"NORMAL"}
ETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM02_Tugas","suhu":30.50,"kelembapan":38.00,"status_led":1,"kondisi_suhu":"NORMAL"}
```



Refleksi:

Hasil dari tugas kali ini saya berhasil untuk menambahkan dan mengedit sourecode yang telah di tentukan berupa Alert kondisi suhu sebagai sebitkut:

```
METRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM02_Tugas","suhu":31.50,"kelembapan":97.30,"status_led":1,"kondisi_suhu":"PANAS"}
METRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM02_Tugas","suhu":32.40,"kelembapan":100.00,"status_led":1,"kondisi_suhu":"PANAS"
```

Alert panas akan keluar jika suhu (>31.0).